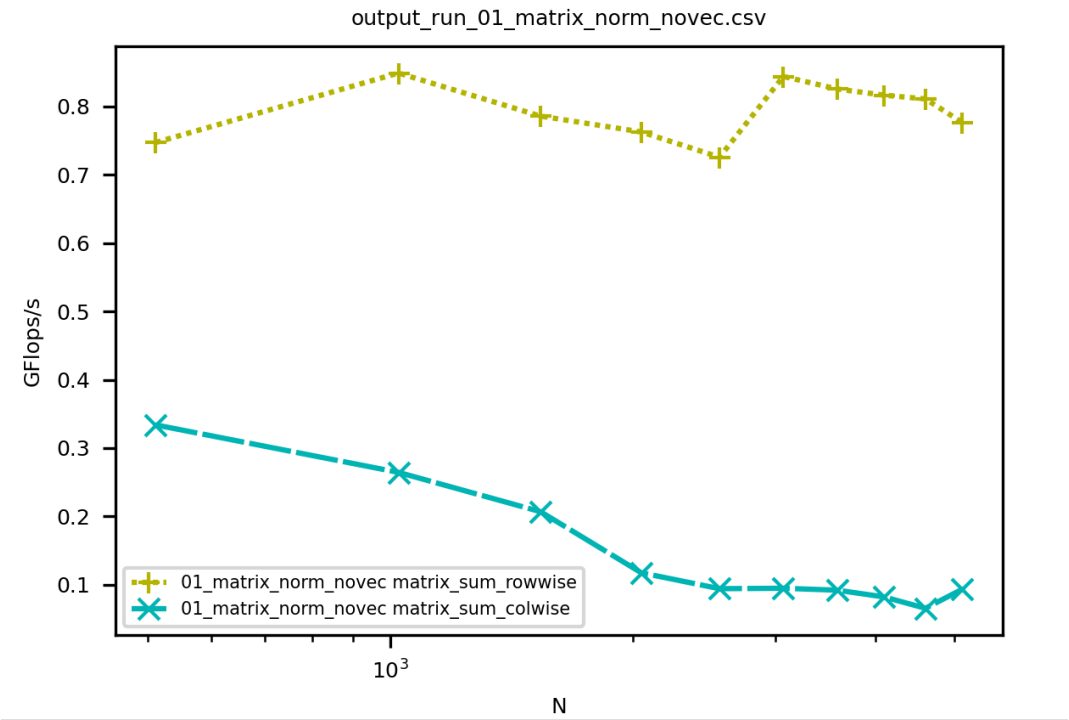
HPC LAB1

Manfred Clase, Samuel Flodin, Felix Lindström

Assignment 1

- Rowwise and colwise norm implemented
- Rowwise executed much faster than colwise (especially for larger matrices)
- The reason for this is Rowwise norm access memory addresses adjacent to each other in memory (spacial memory proximity)

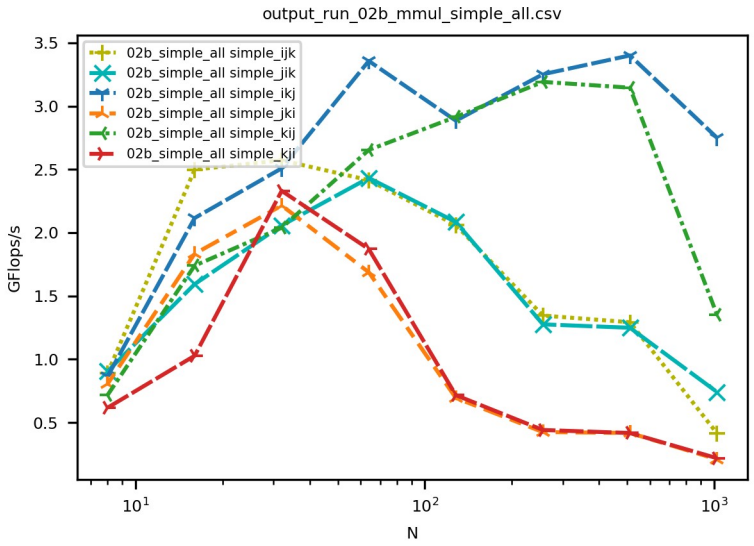


Assignment 2

- The formulas were implemented simply as they were written (with spacial memory access in mind)

Assignment 3

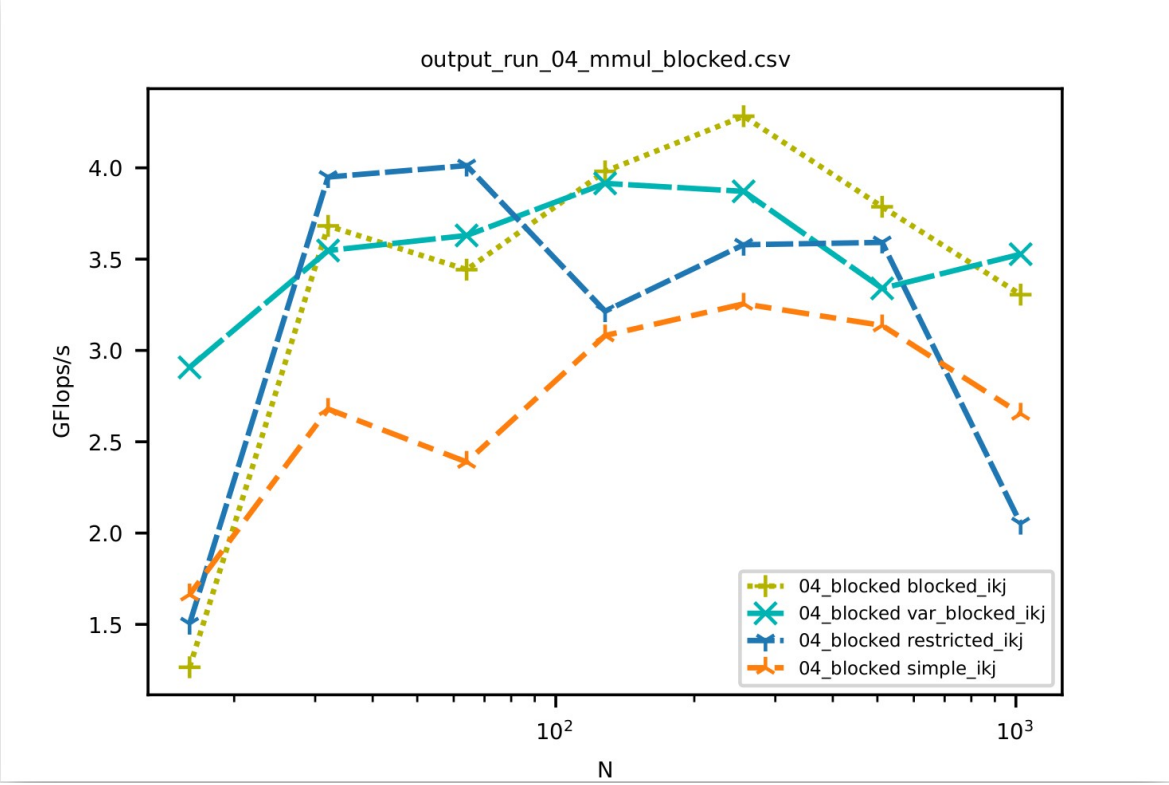
- j should be in the innermost loop, because it only gives row wise iteration.
- i should be in the outermost loop, because it is used for column wise iteration through both the A and C matrices.
- This leaves ikj as the best option and jki as the worst. In our data ikj performed the best and jki and kji the worst.
- For ijk and jik a temporary variable stores the sum. There are therefore less write on C matrix, which seems to be optimal for smaller matrices but scales worse.



Lab 2

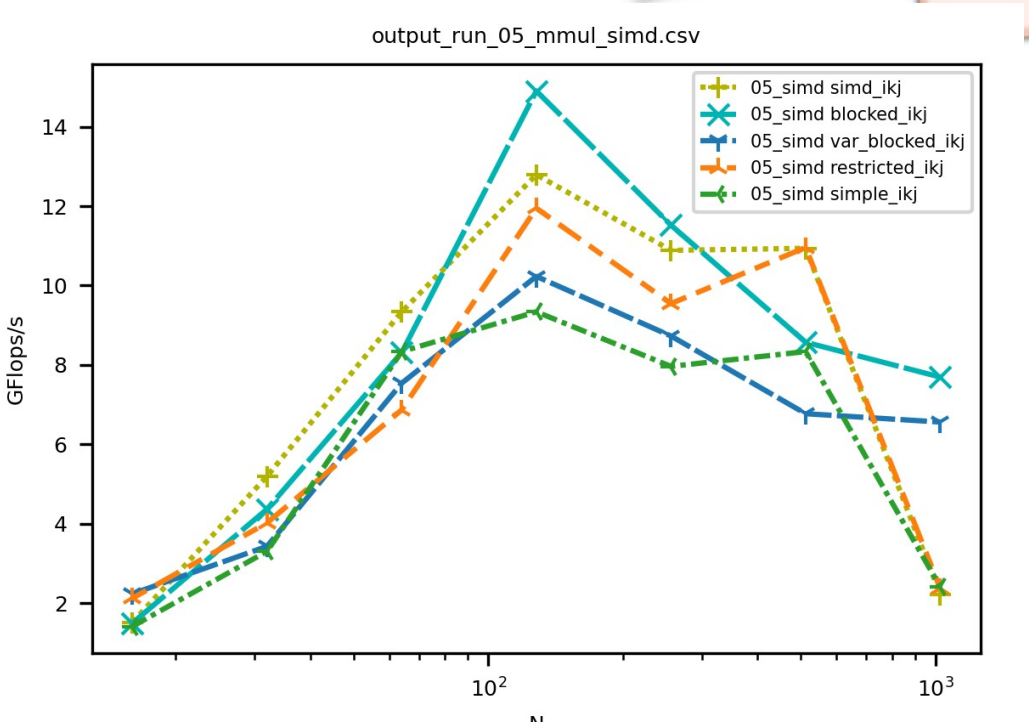
Assignment 1

- In most cases, the cache blocking gives a performance increase, especially for larger values of N.
- Fixed size cache blocking shows slightly better results than variable size cache blocking.



Assignment 2

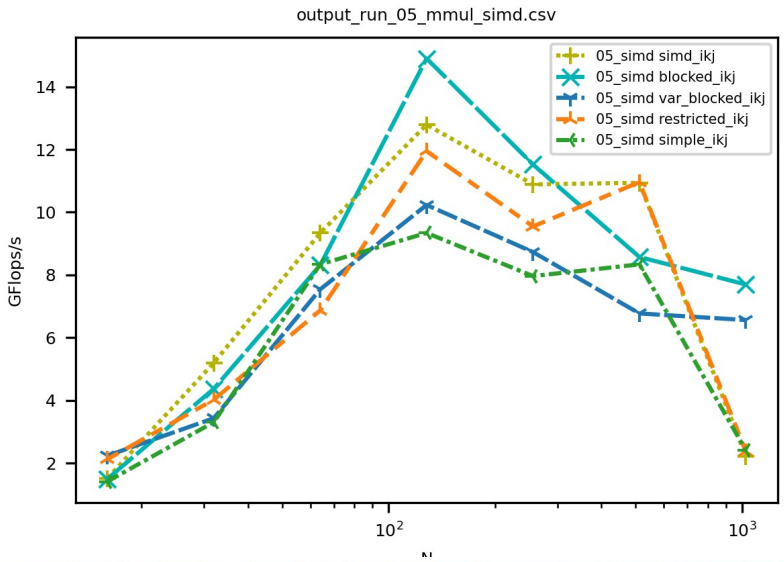
- SIMD outperforms the previous approaches from assignment 1 for most sizes, but not for the largest matrix size N.

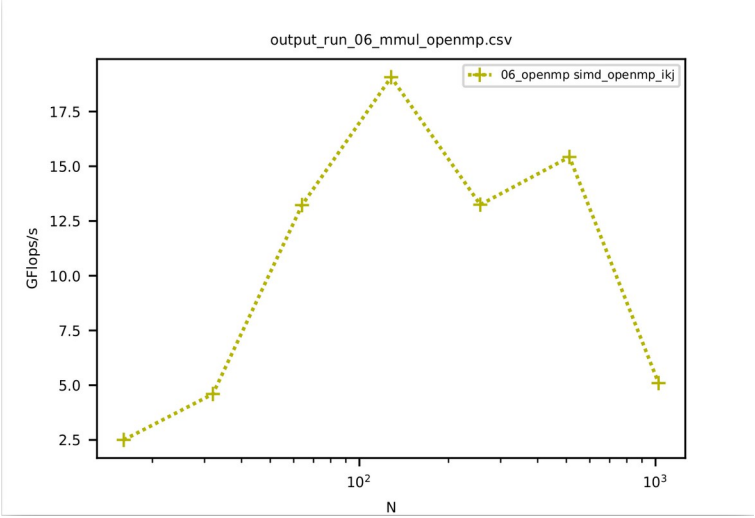


Lab 3

Assignment 1

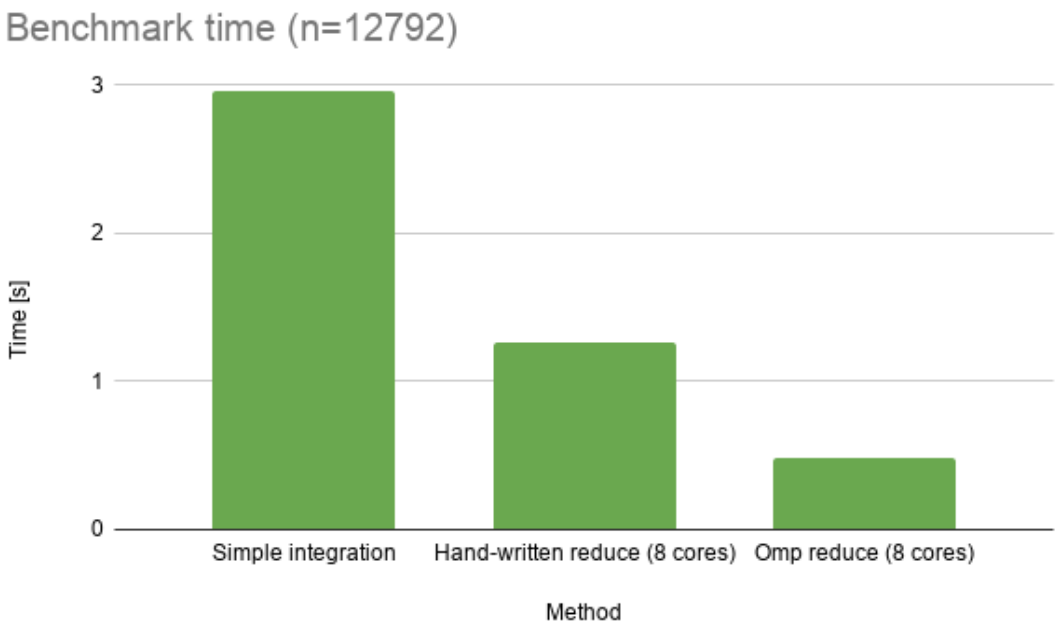
- The OpenMP parallelization gives much better results than the previous versions for greater values N. This is most likely due to utilizing more threads being more impactful than vectorization on the lab computers. Note the different scales on the y axis of the plots.





Assignment 2 : Chosen function

- Function used : f(x) = sin(25 pi/2 x)
- Analytical Integration 0, 1 : 2 / (25 pi)
- n = 12792 needed for accurate approximation

Benchmark of integration approaches

- OpenMP reduction performed the best.
- Simple integration performed worse than parallelization.
- The speedup is not exactly linear with respect to the number of cores because of thread overhead.